

Practical No.3

Implement Min, Max, Sum and Average operations using Parallel Reduction.

Name: Vedant Joshi

Roll No: 24

Code -

```
#include <iostream>

#include <omp.h>

#include <climits>

using namespace std;

void min_reduction(int arr[], int n) {
    int min_value = INT_MAX;

#pragma omp parallel for reduction(min: min_value)
    for (int i = 0; i < n; i++) {
        if (arr[i] < min_value) {
            min_value = arr[i];
        }
    }

    cout << "Minimum value: " << min_value << endl;
}

void max_reduction(int arr[], int n) {
    int max_value = INT_MIN;

#pragma omp parallel for reduction(max: max_value)
    for (int i = 0; i < n; i++) {
        if (arr[i] > max_value) {
            max_value = arr[i];
        }
    }

    cout << "Maximum value: " << max_value << endl;
}

void sum_reduction(int arr[], int n) {
    long long sum = 0;    // Use long long to avoid overflow for large arrays
```

```

#pragma omp parallel for reduction(+: sum)
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    cout << "Sum: " << sum << endl;
}

void average_reduction(int arr[], int n) {
    long long sum = 0;
#pragma omp parallel for reduction(+: sum)
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    if (n > 0) {
        cout << "Average: " << (double)sum / n << endl; // Fixed: divide by n, not (n-1)
    }
    else {
        cout << "Average: 0" << endl;
    }
}

int main() {
    int n;
    cout << "\n=== Parallel Reduction using OpenMP ===\n";
    cout << "Enter total number of elements: ";
    cin >> n;
    if (n <= 0) {
        cout << "Error: Number of elements must be positive!" << endl;
        return 1;
    }
    int* arr = new int[n];
    cout << "Enter " << n << " elements:\n";
    for (int i = 0; i < n; i++) {

```

```

        cin >> arr[i];
    }
    cout << "\n--- Results using Parallel Reduction ---\n";
    min_reduction(arr, n);
    max_reduction(arr, n);
    sum_reduction(arr, n);
    average_reduction(arr, n);
    delete[] arr; // Good practice: free memory
    return 0;
}

```

Output:

vedant@LAPTOP-2AJF2JRG:g++ -fopenmp -o p3 p3.cpp

vedant@LAPTOP-2AJF2JRG:p3.exe

=== Parallel Reduction using OpenMP ===

Enter total number of elements: 5

Enter 5 elements:

20

16

34

59

62

--- Results using Parallel Reduction ---

Minimum value: 16

Maximum value: 62

Sum: 191

Average: 38.2